

Evaluation of Large Language Models in Automated Code Summarization and Documentation: A Systematic Review and Proposed Evaluation Framework

Kubra Aslam

APIIT School of Computing

Colombo, Sri Lanka

Cb012428@students.apiit.lk

Abstract

Despite rapid adoption of Large Language Models (LLMs) for code summarization and documentation generation, evaluation methodology has not kept pace, leaving practitioners without a validated basis for assessing whether a generated documentation is correct. This paper presents a systematic review of the evaluation of LLMs in automated code summarization and documentation, conducted in accordance with the PRISMA 2020 guidelines. Searches across five digital libraries returned 388 records, from which 19 primary studies published between 2023 and 2026 were selected for synthesis. The reviewed evidence shows that LLMs perform strongly on function-level summarization in English-documented Python and Java code, but degrade systematically at class, file and repository granularity, on legacy or obfuscated code, and in non-English settings. It also shows that the field continues to rely on lexical metrics such as BLEU and ROUGE that correlate weakly with expert judgement, that models are poorly calibrated and frequently confident when wrong, and that hallucinated documentation is rarely measured at all. Six research gaps are identified and analyzed. To address them, the paper proposes CoDEval, a four-layer evaluation framework combining a stratified benchmark corpus, a multi-dimensional metric stack incorporating factual-consistency and calibration measures, a structured human rubric with reported inter-rater agreement, and a transparency and reporting protocol. The social, legal and ethical implications of automated documentation, including license provenance, accountability for incorrect documentation and the risk of developer deskilling, are critically evaluated. The paper concludes with a staged validation plan for the proposed framework.

Index Terms—Large language models, code summarization, automated documentation, software engineering, evaluation metrics, hallucination, systematic literature review.

I. INTRODUCTION

Documentation is the connective tissue of a software system. It is what allows an engineer who did not write a module to reason about it, what allows an auditor to establish intent, and what allows a maintenance team to change behavior without reintroducing defects that were fixed years earlier. International standards for software information products treat documentation as a first-class deliverable with its own quality requirements rather than as an optional by-product of development [1]. In practice, however, documentation is the first artefact to be sacrificed under schedule pressure. It is written last, reviewed least, and drifts out of alignment with the code it describes almost immediately, producing a form of technical debt that is difficult to detect because stale documentation looks exactly like current documentation.

The transformer architecture [2] and the subsequent generation of code-trained language models [3] have changed the economics of this problem. Documentation that was previously expensive to produce can now be generated in

seconds, and the practice has moved rapidly from research prototypes into commercial development environments. Surveys of LLM use in software engineering report competitive performance on structured tasks such as code completion and translation, alongside markedly less consistent performance on tasks that require contextual reasoning about intent, of which summarization and documentation are the clearest examples [4], [5].

This creates an asymmetry that motivates the present review. The capability to generate documentation has advanced far faster than the capability to evaluate it. A generated summary that is grammatically fluent, idiomatic and plausible may nevertheless describe behavior the function does not have, and the properties that make LLM output attractive to read are precisely the properties that make incorrect output difficult to detect. Empirical work has shown both that the dominant automated metrics correlate poorly with expert judgement of summary quality [6] and that models are frequently overconfident in exactly those outputs that human raters score lowest [7]. An organization that adopts LLM-generated documentation without a validated evaluation methodology is therefore not merely automating a task; it is industrializing an undetected risk.

This paper addresses that problem through a systematic review of how LLMs are currently evaluated for code summarization and documentation generation, conducted following the PRISMA 2020 reporting guideline [8]. The review is guided by three research questions:

- RQ1: Which models, techniques and prompting strategies are used for LLM-based code summarization and documentation generation, and how are they organized?
- RQ2: How effective are these techniques across programming languages, natural languages and levels of code granularity, and how is that effectiveness measured?
- RQ3: What limitations, research gaps and social, legal and ethical concerns emerge from the current body of evidence?

The contribution of the paper is fourfold. First, it synthesizes a fragmented literature spanning summarization, documentation generation and evaluation methodology into a single account. Second, it identifies and analyses six persistent research gaps and traces each to the evidence that produces it. Third, it proposes CoDEval, a four-layer evaluation framework designed to close those gaps, together with a staged plan for validating it. Fourth, it critically evaluates the social, legal and ethical implications of automated documentation, an area the technical literature has largely left unexamined.

The remainder of the paper is structured as follows. Section II reviews the existing literature across four thematic areas. Section III describes the review methodology, including the search protocol, eligibility criteria and PRISMA workflow. Section IV presents and discusses the findings against each research question and includes a critical cross-study comparison and an evaluation of social, legal and ethical implications. Section V identifies and analyses the research gaps. Section VI proposes the CoDEval framework and its validation plan. Section VII states the limitations of the review, and Section VIII concludes and outlines future work.

II. LITERATURE REVIEW

A. Large Language Models in Software Engineering

The integration of LLMs into software engineering workflows has accelerated considerably, driven by transformer-based architectures pre-trained on large-scale code corpora. Models such as CodeBERT [9], CodeT5 [10] and Code Llama [11] established that bimodal pre-training over paired code and natural language yields representations transferable across a range of downstream tasks. Chen et al. [4] surveyed LLM performance across software engineering tasks and established a pattern that recurs throughout this review: models perform competitively on structurally constrained tasks such as completion, but performance on higher-level cognitive tasks requiring contextual reasoning remains inconsistent and heavily dependent on architecture and available input context. Ghorbani et al. [5] reached a similar conclusion from an industrial perspective, identifying prompt sensitivity, knowledge staleness and the absence of project-specific grounding as the principal constraints on real-world deployment.

Adoption within integrated development environments has demonstrated practical relevance to everyday developer work. Rusum [12] found that context-aware LLM assistance improves productivity in refactoring and documentation tasks, but reported substantial variability in output quality as a function of codebase complexity. That variability is the operative finding. If quality is conditional on properties of the codebase, then any evaluation conducted on a single class of codebase will systematically misestimate performance elsewhere, which is the methodological weakness this review sets out to characterize.

B. Automated Code Summarization

Code summarization, the automated generation of natural language descriptions from source code, has evolved from information retrieval and template-based methods towards deep learning and, more recently, LLM-based approaches [13], [6]. Early neural sequence-to-sequence models improved on retrieval-based methods but were limited by fixed vocabularies and poor cross-language generalization. The release of large paired corpora such as CodeSearchNet [14] marked a significant shift, with subword segmentation and abbreviation expansion subsequently proposed to improve lexical alignment between code tokens and natural language output [13].

Sun et al. [6] conducted one of the most comprehensive evaluations in this area, benchmarking several LLM families across multiple programming languages and abstraction levels. Their results showed that the strongest general-purpose models achieved the highest scores on function-level summarization,

but that all evaluated models degraded significantly on complex multi-module structures. Makharev and Ivanov [15] extended evaluation to class-level and file-level summarization and found that models struggle to maintain semantic coherence as granularity increases. This limitation is compounded by a tendency to produce summaries that are syntactically fluent yet semantically inaccurate. Khatib et al. [16] investigated this through mutation analysis, showing that models frequently fail to reflect meaningful semantic changes in the code within the generated summary. That result is important because it distinguishes surface plausibility from comprehension: a summary that does not change when the code's behavior changes is not describing the code.

The role of prompting has received increasing attention. Fang et al. [17] proposed an enhanced prompting framework and reported BLEU improvements of up to eighteen per cent over zero-shot baselines through structured, context-enriched prompts. Lomshakov et al. [18] developed this further with PROCONSUL, which systematically injects project-level context into prompts and produces measurable improvements across BLEU, ROUGE and BERTScore. Yun et al. [19] showed that few-shot in-context learning with project-specific examples substantially improves performance on project-internal terminology and conventions. Taken together these results establish that prompt design and contextual grounding are first-order determinants of measured performance. This has a consequence the literature does not consistently acknowledge: a reported score describes a model together with a prompting condition, not a model alone, and comparisons that do not hold the prompting condition constant are not comparisons of models.

Multilingual evaluation remains substantially underexplored. Aljohani et al. [20] evaluated a state-of-the-art model on Arabic code summarization and found substantial degradation relative to English settings, attributing this to tokenization mismatches and training-data imbalance. Since the overwhelming majority of published evaluations use English-documented Python and Java, the field's headline performance figures are drawn from its most favorable configuration.

C. Automated Documentation Generation

Documentation generation extends beyond summarization to encompass inline comments, API references, README files and architectural description. Kruse et al. [21] conducted a controlled experiment examining whether developers can effectively prompt LLMs for documentation, concluding that prompt engineering competence significantly influences output quality and that non-expert prompting frequently yields incomplete or misleading documentation. This is a socio-technical finding rather than a purely technical one, and it implies that tool performance in the field is partly a function of user skill distribution rather than model capability.

Diggs et al. [22] evaluated LLMs for legacy code documentation, a domain characterized by undocumented logic and absent modern conventions. They found that plausible documentation is produced for well-structured legacy code but that performance degrades systematically on obfuscated or poorly organized codebases. This is a circular limitation of considerable practical importance: the codebases that most need documentation are the ones for which generated documentation is least reliable. Chakrabarty and Pandey [23] took a different route, fine-tuning on curated README corpora through the

ReadmeReady system and demonstrating that fine-tuned models outperform zero-shot and few-shot baselines, though with reduced cross-domain generalizability.

A clear recent trend is the move towards repository-level documentation. Dhulshette et al. [24] proposed a hierarchical summarization approach and showed that models equipped with hierarchical context representations produce more coherent high-level documentation than those operating on isolated fragments. The DocAgent framework of Yang et al. [25] extended this through a multi-agent architecture that orchestrates generation across repository files, achieving higher coverage and inter-file coherence than single-model approaches. Both studies, however, acknowledge that their evaluation was constrained by the absence of standardized benchmarks for repository-level documentation quality. The field is therefore in the position of building increasingly sophisticated generation systems while the measurement apparatus needed to compare them does not yet exist.

D. Evaluation Frameworks and Metrics

Evaluation in this domain has been dominated by automated metrics inherited from machine translation, principally BLEU [26] and ROUGE [27]. These offer reproducibility and scale, but their adequacy for code-related natural language generation has been widely questioned. Sun et al. [6] argued that BLEU correlates poorly with human judgement of summary quality in software engineering contexts, a finding corroborated by Virk et al. [7], who additionally demonstrated that models are frequently overconfident in low-quality outputs. Zhu et al. [31] found that LLMs underperform relative to smaller fine-tuned models at statement-level summarization, which challenges the assumption that model scale is the primary determinant of output quality.

In response, embedding-based measures such as BERTScore [28] and the code-aware CodeBERTScore [29] have been proposed as complementary signals that better capture meaning-level correspondence, and are increasingly reported alongside lexical metrics [17], [18]. Makharev and Ivanov [15] argued more broadly for code-specific benchmarks that account for the structural properties of source code, on the grounds that general-purpose natural language metrics are insufficient proxies for documentation utility. The viability of LLMs as evaluators was examined by Crupi et al. [30], who found moderate agreement with human judgement but demonstrated susceptibility to positional bias and self-preference when the evaluating and generating models share an architecture. Automating evaluation with the same technology being evaluated therefore reintroduces the bias it was intended to remove.

Across these four areas the same structural weakness recurs. Generation methods are advancing quickly and are evaluated with instruments that their own authors describe as inadequate. Table II summarizes the primary studies included in this review, their principal contributions and their stated limitations.

III. RESEARCH METHODS

This review was conducted as a Systematic Literature Review (SLR) following the PRISMA 2020 statement [8], which specifies a four-stage process of identification, screening, eligibility assessment and inclusion. PRISMA was selected because it makes the selection process auditable and reproducible, which matters in a field where the evidence base

is expanding rapidly and where selective inclusion could easily distort conclusions.

A. Scope and Time Window

The search was restricted to work published between January 2023 and February 2026. This window was chosen deliberately. Instruction-tuned general-purpose LLMs became widely available to developers from late 2022 onwards, and it is only from 2023 that the literature evaluates these models rather than earlier encoder-only or encoder-decoder code models. Extending the window earlier would have introduced a body of work concerned with a materially different class of system.

B. Search Strategy and Keywords

Search terms were organized into three groups covering the model, the task and the evaluation dimension, and combined with Boolean operators:

Model: (“Large Language Model” OR “LLM” OR “GPT” OR “Code Llama” OR “CodeT5” OR “CodeBERT” OR “foundation model”)

Task: (“code summarization” OR “code summarisation” OR “code documentation” OR “comment generation” OR “docstring generation” OR “README generation”)

Evaluation: (“evaluation” OR “benchmark” OR “empirical study” OR “BLEU” OR “ROUGE” OR “human evaluation” OR “hallucination”)

The three groups were joined with AND. Where a database imposed field or length restrictions, the string was decomposed and executed as multiple queries and the results merged before de-duplication.

C. Databases

Searches were executed across Google Scholar, IEEE Xplore, ScienceDirect and ResearchGate. IEEE Xplore and ScienceDirect provide coverage of the peer-reviewed software engineering literature, while Google Scholar was used to capture work indexed outside those two publishers, including recent preprints not yet appearing in a peer-reviewed venue. ResearchGate was searched as a supplementary source, principally to locate author-deposited full texts of records that were otherwise unavailable. Preprints were admitted only where no peer-reviewed version existed and where the work reported primary empirical data. Table I records the yield from each source.

TABLE I
SEARCH RESULTS ACROSS SOURCES

Source	Query focus applied	Records identified
Google Scholar	Model AND Task, first 20 result pages	40
IEEE Xplore	Model AND Task AND Evaluation	27
ScienceDirect	Model AND Task (title, abstract, keywords)	20
ResearchGate	Model AND Task, full-text retrieval	18
Total		105

D. Eligibility Criteria

Studies were included if they were published between 2023 and 2026, evaluated one or more LLMs on a code summarization or documentation generation task, reported

primary empirical results in the form of automated metric scores or human assessment, and were written in English. Studies were excluded if they contained no LLM component, or if they addressed a different task such as code generation, defect prediction or program repair rather than summarization or documentation. Duplicate preprints of an included peer-reviewed paper, secondary reviews without primary data, and records not available in full text were also removed.

E. PRISMA Workflow

Searching across the four sources identified 105 records. Removal of four duplicates left 101 unique records for screening. Title and abstract screening excluded a further six records as not relevant to the topic, leaving 95 articles for full-text assessment. Of these, 76 were excluded with reasons recorded: 27 contained no LLM component, 40 addressed code generation or another software engineering task rather than summarization or documentation, and 9 reported no evaluation data. Nineteen studies satisfied all criteria and were carried into the synthesis. Fig. 1 presents the complete flow.

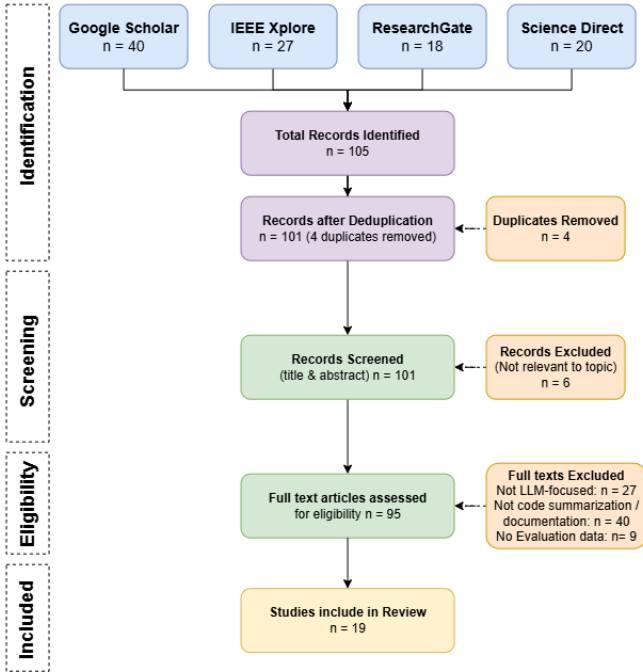


Fig. 1. PRISMA 2020 flow diagram of the study selection process.

F. Data Extraction and Quality Appraisal

Data were extracted into a standardized form capturing the models evaluated, programming and natural languages covered, code granularity, dataset provenance, prompting condition, evaluation metrics, presence and design of human evaluation, and reported limitations. Quality was appraised narratively against four criteria: clarity of the evaluation protocol, robustness and provenance of the dataset, whether human assessment was included, and reproducibility of the reported procedure. Studies were assigned to high, medium or low quality bands, and no study was excluded on quality grounds alone; the band was instead used to weigh the confidence attached to each finding during synthesis.

G. Synthesis Approach

Because the included studies differ substantially in models, datasets, metrics and reporting conventions, quantitative meta-analysis was not appropriate and a narrative synthesis was used. Studies were grouped into four thematic categories corresponding to the structure of Section II. Fig. 2 shows the distribution of the corpus across those categories and across years of publication, and illustrates two properties of the evidence base: it is heavily concentrated in the two most recent years, and summarization is better represented than documentation generation or evaluation methodology.

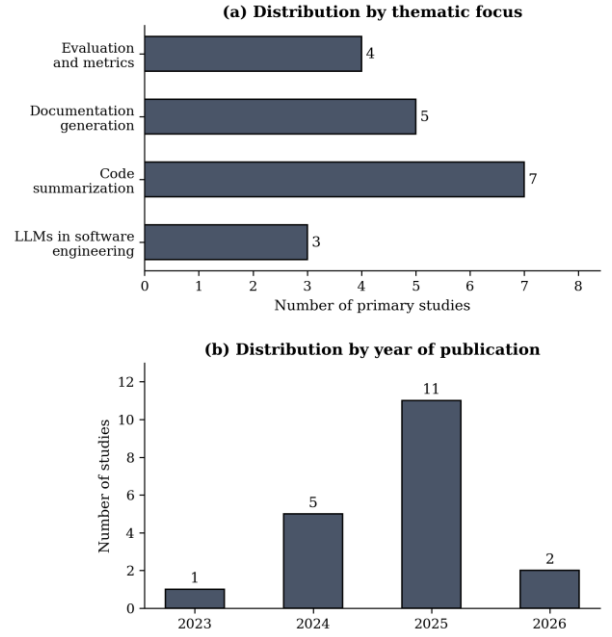


Fig. 2. Distribution of the 19 included primary studies by thematic focus and year of publication.

TABLE II
OVERVIEW OF INCLUDED PRIMARY STUDIES

Ref.	Focus and principal contribution	Stated limitation
[4]	Survey of LLM evaluation across code tasks	Secondary evidence; no unified protocol proposed
[5]	Industrial survey of LLM code generation	Deployment constraints reported qualitatively
[6]	Multi-model, multi-language summarization benchmark	Degrades on multi-module code; BLEU–human mismatch
[7]	Calibration of LLMs on code summarization	Confidence poorly aligned with quality
[12]	Context-aware LLM assistance inside IDEs	Quality varies with codebase complexity
[13]	Abbreviation expansion and subword segmentation	Lexical focus; limited semantic assessment
[15]	Summarization beyond function level	Coherence falls at class and file granularity
[16]	Mutation analysis of summary sensitivity	Summaries insensitive to semantic change
[17]	Enhanced prompting framework (+18% BLEU)	Gains measured on lexical metrics only
[18]	PROCONSUL project-context injection	Context budget and cost not analysed
[19]	Project-specific few-shot in-context learning	Requires curated in-project examples
[20]	Arabic code summarization evaluation	Marked degradation outside English
[21]	Controlled experiment on developer prompting	Output quality depends on prompting skill

[22]	LLM documentation of legacy code	Fails on obfuscated or unstructured code
[23]	ReadmeReady fine-tuning for README generation	Reduced cross-domain generalisability
[24]	Hierarchical repository-level summarization	No standard benchmark for repo-level quality
[25]	DocAgent multi-agent documentation generation	Evaluation constrained by missing benchmarks
[30]	LLM-as-a-judge for summarization	Positional bias and self-preference effects
[31]	Statement-level summarization effectiveness	LLMs underperform smaller fine-tuned models

IV. RESULTS AND DISCUSSION

RQ1: Which models, techniques and prompting strategies are used for LLM-based code summarization and documentation generation, and how are they organized?

The 19 included studies employ five broad classes of approach, summarized in Table III. The first is direct prompting of general-purpose instruction-tuned LLMs, which is the most common configuration and serves as the de facto baseline in most papers [6], [15], [20]. The second is the use of code-specific pre-trained models, either encoder-decoder architectures or code-tuned decoder models, evaluated as comparators [4], [31]. The third is prompt engineering, in which the generation prompt is enriched with structure, exemplars or reasoning scaffolding [17], [19]. The fourth is context augmentation, in which project or repository-level information is retrieved and injected into the model context [18], [24]. The fifth is agentic orchestration, in which multiple model invocations are coordinated across files to produce documentation with inter-file coherence [25].

Two observations follow. First, the trajectory of the field is unmistakably away from the model as the unit of improvement and towards the surrounding system: what context is retrieved, how the prompt is constructed, and how invocations are orchestrated. Second, and less comfortably, evaluation practice has not followed that trajectory. Most studies still report a score attached to a model name, when the entity actually evaluated is a pipeline consisting of a model, a context-retrieval policy and a prompt template. This mismatch between what is built and what is measured is the first structural weakness in the literature.

TABLE III
TAXONOMY OF APPROACHES IDENTIFIED (RQ1)

Approach class	Mechanism	Studies
Zero-shot prompting	General-purpose LLM invoked directly on the code unit	[6], [15], [20]
Code-specific models	Encoder-decoder or code-tuned models used as comparators	[4], [31]
Prompt engineering	Structured, exemplar-rich or reasoning-scaffolded prompts	[17], [19], [21]
Context augmentation	Retrieval and injection of project or repository context	[18], [24]
Agentic orchestration	Multiple coordinated invocations across repository files	[25]
Fine-tuning	Supervised adaptation on curated documentation corpora	[23]

RQ2: How effective are these techniques across programming languages, natural languages and levels of code granularity, and how is that effectiveness measured?

Effectiveness is strongly conditional, and the conditions matter more than the aggregate figures. Table IV consolidates the reported evidence by generation condition.

At function level, on English-documented Python and Java, the strongest general-purpose models perform well and are consistently reported as the best performers in head-to-head comparisons [6]. This is the configuration in which most published figures are obtained, and it is also the configuration least representative of industrial software. As granularity increases the picture reverses. Class-level and file-level summarization shows loss of semantic coherence [15], and repository-level documentation is coherent only when hierarchical context representations or multi-agent orchestration are supplied [24], [25]. At the opposite extreme, statement-level summarization is performed better by smaller fine-tuned models than by large general-purpose ones [31], which is a useful corrective to the assumption that capability scales monotonically with parameter count.

Prompting and context are reliable sources of gain. Structured prompting yields BLEU improvements reported at up to eighteen per cent [17], project-context injection improves lexical and semantic scores concurrently [18], and project-specific few-shot examples substantially improve handling of internal terminology [19]. Against this, code properties impose hard limits: obfuscated and unstructured legacy code degrades output systematically [22], and non-English settings degrade it further through tokenization mismatch and training-data imbalance [20].

The measurement side is weaker than the generation side. BLEU and ROUGE remain the most frequently reported metrics despite explicit findings that they correlate poorly with expert judgement [6], [7]. Embedding-based measures such as BERTScore and CodeBERTScore appear in a minority of studies as supplementary signals [17], [18]. Human evaluation, where present, is typically conducted with small rater pools and without reported inter-rater agreement, which makes the resulting scores difficult to interpret and impossible to compare across papers. Only two studies in the corpus attempt to measure semantic faithfulness directly, one through mutation sensitivity [16] and one through calibration analysis [7]. The consequence is that the property practitioners care about most, whether the documentation is true, is the property least frequently measured.

TABLE IV
REPORTED EFFECTIVENESS BY GENERATION CONDITION (RQ2)

Condition	Reported outcome	Studies
Function level, English, Python/Java	Strongest performance; frequent baseline for headline figures	[6], [17]
Class and file level	Loss of semantic coherence as granularity rises	[15]
Repository level	Acceptable only with hierarchical or multi-agent context	[24], [25]
Statement level	Smaller fine-tuned models outperform large LLMs	[31]
Enriched prompting	Up to 18% BLEU gain over zero-shot baselines	[17], [19]
Project context injected	Concurrent gains in lexical and semantic scores	[18]
Legacy or obfuscated code	Systematic degradation of documentation quality	[22]
Non-English settings	Substantial degradation; tokenization mismatch	[20]

RQ3: What limitations, research gaps and social, legal and ethical concerns emerge from the current body of evidence?

Five categories of limitation recur across the corpus and are consolidated in Table V. Technically, the dominant issue is hallucination: fluent output that misdescribes behavior, evidenced by insensitivity to code mutation [16] and by overconfidence in low-quality generations [7]. Methodologically, the dominant issue is metric validity, since the field's most reported numbers are produced by instruments its own authors have shown to be weakly correlated with the judgement they are meant to approximate [6]. Structurally, generalizability is limited by the concentration of evaluation on a narrow slice of languages and granularities [15], [20]. Operationally, cost, context-window budget and model version drift are rarely reported at all, which makes replication difficult even when data are available. Finally, the evaluation infrastructure itself is contested, since delegating judgement to LLMs reintroduces bias through positional and self-preference effects [30].

TABLE V
CHALLENGES AND LIMITATIONS IDENTIFIED (RQ3)

Category	Limitation	Studies
Technical	Hallucinated documentation; insensitivity to semantic change	[16], [7]
Methodological	Lexical metrics correlate weakly with expert judgement	[6], [7]
Structural	Narrow language and granularity coverage	[15], [20]
Operational	Cost, context budget and version drift unreported	[18], [5]
Evaluative	LLM judges show positional and self-preference bias	[30]
Human factors	Output quality contingent on user prompting skill	[21]

A. Critical Cross-Study Comparison

Read individually, the included studies present a picture of steady progress. Read against one another, they present something more equivocal. The studies reporting the strongest results are, with few exceptions, those evaluating at the most favorable granularity on the most favorable languages using the metrics least sensitive to semantic error. The studies reporting the weakest results are those that deliberately stress one of those variables: granularity [15], language [20], code quality [22], or semantic faithfulness [16]. Performance in this literature is therefore not a stable property of a model but a function of how hard the evaluation is willing to push.

A second tension concerns the direction of investment. Context augmentation and agentic orchestration produce real gains [18], [24], [25], but each adds retrieval, coordination and token cost that the evaluations do not price. A framework that improves coherence at several times the inference cost may or may not be preferable to a cheaper approach, and the literature currently provides no basis for that judgement because cost is almost never reported alongside quality.

A third tension is evidential. Fine-tuning improves in-domain performance at the expense of transfer [23], while

prompting and context injection improve transfer without permanently changing the model [17], [18]. These are genuinely different trade-offs, but because the two lines of work use different datasets and different metrics they have never been compared under equivalent conditions. The disagreement in the literature is not resolved by evidence; it is preserved by incompatible measurement.

B. Social, Legal and Ethical Implications

The technical literature reviewed here treats documentation generation almost exclusively as an engineering problem. The implications extend considerably further, and a critical evaluation of an emerging technology is incomplete without them.

1) Provenance, licensing and intellectual property:

Code-trained models are pre-trained on large corpora of publicly hosted source code carrying heterogeneous licenses, including copyleft terms. Documentation generated from such a model may reproduce descriptive text, structure or identifier conventions derived from licensed material, and the provenance of that text is not traceable from the output. Organizations that commit generated documentation into a proprietary repository therefore accept a licensing exposure they cannot easily quantify. None of the studies reviewed reports licence provenance for training data or generated output.

2) Confidentiality and data protection:

Generating documentation for a proprietary codebase using a hosted model requires transmitting that code, and often surrounding project context [18], to a third-party service. Where the code processes personal data, or where comments and test fixtures embed personal data, this constitutes processing under data protection law and requires a lawful basis, a processor agreement and consideration of transfer restrictions [33]. Retrieval-augmented and agentic approaches increase exposure precisely because their advantage derives from sending more context.

3) Accountability for incorrect documentation:

Incorrect documentation is not a neutral failure. It actively misleads the engineer who relies on it, and in regulated or safety-related domains documentation forms part of the audit and certification record. Because generated documentation is fluent and confident, and because models are demonstrably overconfident when wrong [7], the ordinary human signals that prompt skepticism are absent. Responsibility remains with the developer who accepts the output, yet the evidence indicates that developers are poorly positioned to detect the failures, particularly where prompting competence is limited [21]. Emerging regulation of AI systems places transparency and human oversight obligations on deployers [32], and documentation tooling used in high-risk contexts is unlikely to remain outside that scope indefinitely.

4) Deskilling and epistemic dependence:

Writing documentation forces an engineer to articulate intent, and that articulation frequently exposes design problems. Automating the artefact removes the cognitive step that produced the benefit. If comprehension is progressively delegated to a system that has been shown not to track semantic change reliably [16], teams may acquire documentation while losing the understanding it was supposed to represent.

5) Equity and linguistic exclusion:

Performance degradation outside English [20] means the benefits of this technology are distributed unevenly along existing linguistic lines. Developers working in non-English contexts receive lower-quality assistance from the same tools, and because evaluation is itself concentrated on English the deficit remains largely unmeasured. This is a social consequence produced directly by a methodological choice, which is why it belongs in an evaluation framework rather than in a closing paragraph.

V. GAP IDENTIFICATION AND ANALYSIS

Six gaps follow from the synthesis. They are stated here with the evidence that produces each and the consequence of leaving it unaddressed.

G1: No unified evaluation framework. No study in the corpus integrates automated metrics, semantic measures and structured human assessment into a single reproducible protocol. Each paper constructs its own instrument, so results cannot be compared across papers and apparent disagreements cannot be resolved [6], [15], [30].

G2: Function-level bias. Evaluation is concentrated at function level while practitioner need is concentrated at class, file and repository level, where performance is known to degrade [15], [24], [25]. Reported performance therefore overstates real-world utility in a direction that is systematic rather than random.

G3: Monolingual and narrow-language evaluation. Coverage is dominated by English documentation of Python and Java [20]. Generalization claims made from this base are unsupported, and the resulting quality gap falls on developers already less well served.

G4: Hallucination is generated but not measured. Fluent, semantically inaccurate documentation is widely acknowledged yet rarely quantified; only mutation-based [16] and calibration-based [7] approaches address it, and neither has been adopted as standard practice.

G5: Metric-judgement misalignment. The most reported metrics are the least valid, and the alternatives are either not code-aware or, in the case of LLM judges, carry their own bias [6], [29], [30].

G6: Absent socio-technical and legal dimensions. No study reports license provenance, data-protection posture, cost, or the human factors that condition output quality in deployment [5], [21]. Evaluation stops at the metric, while the risks that determine whether a tool can responsibly be adopted lie beyond it.

These gaps are connected rather than independent. G1 sustains G2 and G3, because in the absence of a shared protocol each study chooses its own scope and every choice tends towards the tractable. G5 sustains G4, because a measurement apparatus insensitive to semantic error cannot detect hallucination even in principle. Addressing them piecemeal is therefore unlikely to succeed, which motivates the integrated proposal that follows.

VI. PROPOSED SOLUTION: THE CODEVAL FRAMEWORK

This paper proposes CoDEval, a Code Documentation Evaluation Framework consisting of four layers, shown in Fig. 3. The design principle is that documentation quality is not a

scalar and should not be reported as one. CoDEval produces a quality profile across dimensions, so that a model that is fluent but unfaithful is distinguishable from one that is accurate but terse, a distinction current single-score reporting erases.

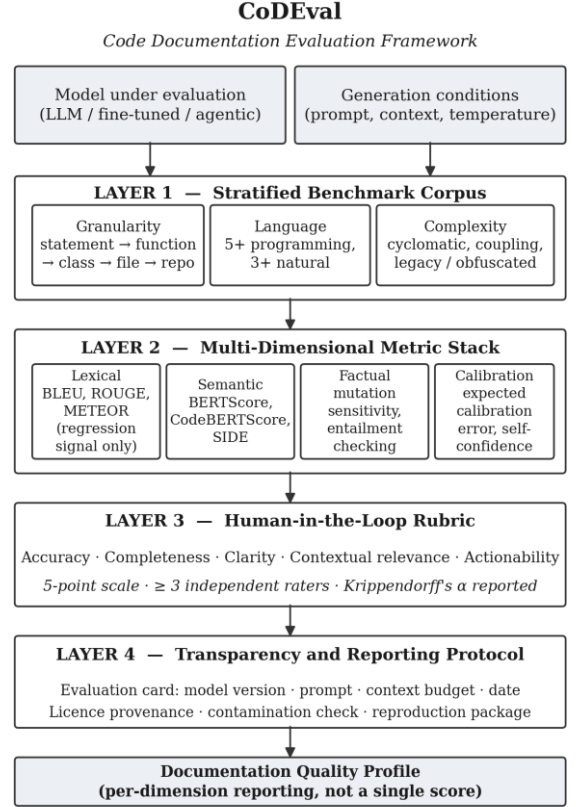


Fig. 3. The proposed CoDEval framework. Each layer addresses one or more of the gaps identified in Section V.

A. Layer 1: Stratified Benchmark Corpus

The corpus is stratified along three axes rather than sampled uniformly. Granularity spans statement, function, class, file and repository units, so that degradation with granularity becomes visible in the result rather than hidden by the sampling frame. Language covers at least five programming languages spanning paradigms, and at least three natural languages for identifiers, comments and reference documentation, directly addressing G3. Complexity is stratified using cyclomatic complexity, coupling and a legacy or obfuscation flag, so that the codebases which most need documentation are represented rather than filtered out. Reference documentation is drawn from human-maintained sources with recorded licence terms, and every item carries a contamination check against known public training corpora. This layer addresses G2 and G3.

B. Layer 2: Multi-Dimensional Metric Stack

Four metric families are computed and reported separately. Lexical metrics (BLEU, ROUGE, METEOR) are retained but demoted to regression signals, useful for detecting change between versions of a system and not treated as evidence of quality, which follows directly from the correlation findings of [6] and [7]. Semantic metrics (BERTScore, CodeBERTScore) capture meaning-level correspondence and are code-aware [28], [29]. Factual-consistency metrics operationalize hallucination detection through two mechanisms: a mutation sensitivity score, which applies behavior-changing mutations to

the source unit and measures whether the regenerated summary changes accordingly, following [16]; and entailment checking, in which each claim in the generated documentation is tested against static properties extractable from the code, such as parameters, return types, raised exceptions and side effects. Calibration metrics record expected calibration error between self-reported confidence and measured quality, following [7]. This layer addresses G4 and G5.

C. Layer 3: Human-in-the-Loop Rubric

Automated measures cannot establish whether documentation is useful, so a structured human protocol is specified rather than left to each study. Five dimensions are rated on a five-point scale: accuracy, completeness, clarity, contextual relevance and actionability. A minimum of three independent raters with stated experience levels is required, and Krippendorff's alpha is reported so that agreement can be judged rather than assumed. Rater experience is recorded as a variable because [21] shows that human factors materially affect outcomes. Where LLM judges are used to scale the protocol, [30] requires that the judging model differ from the generating model and that positional bias be controlled through order randomization. This layer addresses G1 and G5.

D. Layer 4: Transparency and Reporting Protocol

Each evaluation emits a structured evaluation card recording model identifier and version, exact prompt template, retrieval policy and context budget, decoding parameters, evaluation date, measured token cost, license provenance of both corpus and outputs, and the contamination check result. This turns the pipeline rather than the model into the declared unit of evaluation, which is what the results in Section IV indicate are actually being measured. It also brings cost, provenance and data-protection posture inside the evaluation boundary. This layer addresses G1 and G6.

E. Mapping to the Identified Gaps

Table VI maps each gap to the responsible component and the expected effect.

TABLE VI
MAPPING OF RESEARCH GAPS TO CODEVAL COMPONENTS

Gap	Component	Expected effect
G1 No unified framework	Layers 1–4 combined	Cross-study comparison becomes possible under a declared protocol
G2 Function-level bias	Layer 1 granularity axis	Degradation with granularity is reported rather than hidden
G3 Narrow language coverage	Layer 1 language axis	Non-English and multi-paradigm deficits become measurable
G4 Unmeasured hallucination	Layer 2 factual family	Semantic infidelity is quantified, not inferred from fluency
G5 Metric misalignment	Layers 2 and 3	Lexical scores demoted; human agreement reported
G6 Socio-technical blind spot	Layer 4 evaluation card	Cost, licence and data-protection posture enter the record

F. Validation Plan

CoDEval is proposed as a framework and requires validation in three stages. The first is a construct pilot, applying the metric stack to an existing benchmark and testing whether the factual-

consistency family separates outputs that lexical metrics score identically. The second is a reliability study, in which the human rubric is applied by multiple rater cohorts to establish inter-rater agreement and to estimate how far LLM judging can substitute for human rating without loss of validity. The third is an industrial case study applying the full framework to a live codebase, comparing framework scores with downstream outcomes such as time to comprehension, defect rates in maintenance tasks and reviewer acceptance. The third stage is the decisive one, since a framework that predicts nothing about practice would replicate the very weakness it was designed to correct.

G. Implementation Challenges

Four challenges are acknowledged. Human rating is expensive and constrains the achievable corpus size, which is why the rubric specifies a minimum rather than an ideal rater count. Model version drift means results carry an expiry date, which the evaluation card records but cannot prevent. Benchmark contamination is a persistent hazard, since any published corpus may enter subsequent training data, so the contamination check must be re-run rather than performed once. Finally, entailment checking depends on static analysis support that is more mature for some languages than others, which risks reintroducing the language bias the framework is designed to expose; the practical mitigation is to report entailment coverage per language rather than to aggregate across them.

VII. LIMITATIONS OF THIS REVIEW

Four limitations should be noted. The review covers English-language publications from 2023 to 2026 across five databases, so relevant work in other languages, in grey literature or in industrial practice may have been missed, and the exclusion of pre-2023 work means encoder-only and encoder-decoder approaches are represented only where recent studies use them as comparators. The corpus of 19 studies is modest, reflecting a deliberately narrow focus on work that both applies LLMs to summarization or documentation and reports primary evaluation data. Screening and quality appraisal were performed by a single reviewer, so inter-reviewer agreement could not be established and selection bias cannot be excluded. Finally, CoDEval is a conceptual proposal supported by the reviewed evidence but not yet empirically validated, and the expected effects in Table VI are argued rather than demonstrated.

VIII. CONCLUSION AND FUTURE WORK

This review examined how LLMs are evaluated for automated code summarization and documentation, synthesizing 19 primary studies selected through a PRISMA-guided protocol. The evidence shows a field advancing rapidly in generation capability while measuring that capability with instruments its own authors describe as inadequate. Performance is strong at function level in English-documented Python and Java, and degrades systematically with granularity, with code quality and outside English. Prompt design and injected project context are first-order determinants of measured performance, which means published scores describe pipelines rather than models. Most consequentially, the property practitioners most need, whether generated

documentation is true, is the least often measured property: models are insensitive to behavioral change in the code they describe and are overconfident precisely where they are least accurate.

Six gaps were identified and analyzed: the absence of a unified evaluation framework, function-level bias, narrow language coverage, unmeasured hallucination, misalignment between metrics and expert judgement, and the absence of socio-technical and legal considerations from evaluation practice. Because these gaps sustain one another, the paper proposed CoDEval, an integrated four-layer framework combining a stratified benchmark corpus, a multi-dimensional metric stack incorporating factual-consistency and calibration measures, a structured human rubric with reported agreement, and a transparency and reporting protocol that brings cost, provenance and data-protection posture inside the evaluation boundary.

Future work follows the three-stage validation plan of Section VI-F, beginning with a construct pilot of the factual-consistency family on an existing benchmark and proceeding to an industrial case study linking framework scores to downstream comprehension and maintenance outcomes. Three further directions are worth pursuing. First, the construction of a genuinely multilingual documentation benchmark, which is a prerequisite for progress on G3 and which has no current dataset supplies. Second, the development of cost-aware evaluation, so that the gains from context augmentation and agentic orchestration can be weighed against the resources they consume. Third, longitudinal study of documentation drift, since the value of generated documentation depends not only on whether it is correct when written but on whether it remains correct as the code evolves, a question the current literature does not address at all.

REFERENCES

- [1] ISO/IEC/IEEE 26514:2022, “Systems and software engineering — Design and development of information for users,” International Organization for Standardization, 2022, doi: <https://www.iso.org/standard/77451.html>
- [2] A. Vaswani et al., “Attention is all you need,” in Proc. 31st Int. Conf. Neural Information Processing Systems (NIPS), Long Beach, CA, USA, 2017, pp. 6000–6010, doi: <https://arxiv.org/pdf/1706.03762>
- [3] M. Chen et al., “Evaluating large language models trained on code,” arXiv:2107.03374, 2021, doi: <https://arxiv.org/pdf/2107.03374>
- [4] L. Chen et al., “A survey on evaluating large language models in code generation tasks,” arXiv:2408.16498, 2025, doi: <https://arxiv.org/pdf/2408.16498>
- [5] M. Ghorbian, M. Ghobaei-Arani, and A. Souri, “Large language models for code generation: A survey,” *Computer Standards & Interfaces*, vol. 95, 2026.
- [6] W. Sun et al., “Source code summarization in the era of large language models,” in Proc. IEEE/ACM 47th Int. Conf. Software Engineering (ICSE), Ottawa, ON, Canada, 2025, doi: <https://arxiv.org/pdf/2407.07959>
- [7] Y. Virk, P. Devanbu, and T. Ahmed, “Calibration of large language models on code summarization,” *ACM Trans. Software Engineering and Methodology*, 2025, doi: <https://dl.acm.org/doi/epdf/10.1145/3729400>
- [8] M. J. Page et al., “The PRISMA 2020 statement: An updated guideline for reporting systematic reviews,” *BMJ*, vol. 372, p. n71, Mar. 2021, doi: <https://www.bmj.com/content/bmj/372/bmj.n71.full.pdf>
- [9] Z. Feng et al., “CodeBERT: A pre-trained model for programming and natural languages,” in Findings of EMNLP 2020, pp. 1536–1547, doi: <https://arxiv.org/pdf/2002.08155>
- [10] Y. Wang, W. Wang, S. Joty, and S. C. H. Hoi, “CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation,” in Proc. EMNLP, 2021, pp. 8696–8708, doi: <https://arxiv.org/pdf/2109.00859>
- [11] B. Rozière et al., “Code Llama: Open foundation models for code,” arXiv:2308.12950, 2023, doi: <https://arxiv.org/pdf/2308.12950>
- [12] G. P. Rusum, “Large language models in IDEs: Context-aware coding, refactoring, and documentation,” *Int. J. Emerging Trends in Computer Science and Information Technology*, vol. 4, no. 2, 2023.
- [13] Y.-G. Liang et al., “Automatic code summarization using abbreviation expansion and subword segmentation,” *Expert Systems*, 2024.
- [14] H. Husain, H.-H. Wu, T. Gazit, M. Allamanis, and M. Brockschmidt, “CodeSearchNet challenge: Evaluating the state of semantic code search,” arXiv:1909.09436, 2019.
- [15] V. Makharev and V. Ivanov, “Code summarization beyond function level,” in Proc. IEEE/ACM Int. Workshop on Large Language Models for Code (LLM4Code), 2025.
- [16] L. Khatib, M. Papadakis, B. Vaziri, and M. Nejati, “Examining LLMs’ ability to summarize code through mutation analysis,” arXiv, 2026.
- [17] M. Fang et al., “Enhanced prompting framework for code summarization with large language models,” *ACM Trans. Software Engineering and Methodology*, vol. 34, no. 2, 2025.
- [18] V. Lomshakov et al., “PROCONSUL: Project context for code summarization with LLMs,” in Proc. EMNLP 2024 Industry Track, pp. 866–890.
- [19] S. Yun, S. Lin, X. Gu, and B. Shen, “Project-specific code summarization with in-context learning,” *J. Systems and Software*, vol. 216, 2024.
- [20] A. Aljohani, R. Alharbi, A. Alqahtani, and W. Alshehri, “Evaluating LLMs for Arabic code summarization: Challenges and insights from GPT-4,” in Proc. 8th Int. Conf. Data Science and Machine Learning Applications (CDMA), 2025.
- [21] H.-A. Kruse, T. Puhlfürß, and W. Maalej, “Can developers prompt? A controlled experiment for code documentation generation,” in Proc. IEEE Int. Conf. Software Maintenance and Evolution (ICSME), 2024.
- [22] C. Digges et al., “Leveraging LLMs for legacy code modernization: Evaluation of LLM-generated documentation,” in Proc. IEEE/ACM Int. Workshop on Large Language Models for Code (LLM4Code), 2025.
- [23] S. Chakrabarty and S. Pandey, “ReadmeReady: Free and customizable code documentation with LLMs — a fine-tuning approach,” *J. Open Source Software*, 2025.
- [24] N. Dhulshette, S. Sharma, and V. Kumar, “Hierarchical repository-level code summarization for business applications using local LLMs,” in Proc. IEEE/ACM Int. Workshop on Large Language Models for Code (LLM4Code), 2025.
- [25] D. Yang et al., “DocAgent: A multi-agent system for automated code documentation generation,” in Proc. 63rd Annual Meeting of the Association for Computational Linguistics (ACL), vol. 3, 2025.
- [26] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, “BLEU: A method for automatic evaluation of machine translation,” in Proc. 40th Annual Meeting of the ACL, 2002, pp. 311–318.
- [27] C.-Y. Lin, “ROUGE: A package for automatic evaluation of summaries,” in Text Summarization Branches Out, ACL Workshop, 2004, pp. 74–81.
- [28] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, “BERTScore: Evaluating text generation with BERT,” in Proc. Int. Conf. Learning Representations (ICLR), 2020.
- [29] S. Zhou, U. Alon, S. Agarwal, and G. Neubig, “CodeBERTScore: Evaluating code generation with pretrained models of code,” in Proc. EMNLP, 2023, pp. 13921–13937.
- [30] G. Crupi, R. Tufano, A. Vitale, A. Mastropaolo, D. Poshyanyk, and G. Bavota, “On the effectiveness of LLM-as-a-judge for code generation and summarization,” *IEEE Trans. Software Engineering*, vol. 51, no. 8, 2025.
- [31] J. Zhu, Y. Miao, T. Xu, J. Zhu, and X. Sun, “On the effectiveness of large language models in statement-level code summarization,” in Proc. IEEE 24th Int. Conf. Software Quality, Reliability and Security (QRS), 2024.
- [32] European Parliament and Council, “Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act),” *Official Journal of the European Union*, 2024.
- [33] European Parliament and Council, “Regulation (EU) 2016/679 on the protection of natural persons with regard to the processing of personal data (General Data Protection Regulation),” *Official Journal of the European Union*, 2016.